STUDENT OLYMPIAD “STEP INTO FUTURE”, ENGINEERING

№ 44854

Section: Cybersecurity and digital criminology

Using Computer Vision technologies to increase forensic sketches accuracy

Author: Chekueva Alima
Olympus, 10th grade

Moscow-2024

ANNOTATION

During the work, possible solutions to the problem of low recognition of photofits using computer vision were studied. The goal was to find a mechanism that corrects the shortcomings of the currently used software.

Of the two possible options, one was implemented - using image stylization technology. Different style samples were selected, but the best result was shown by the face model, which had averaged features of both genders. The input images were shifted to the left relative to the background and reformatted to a size of 128x128 pixels. The number of iterations was 300. The program was written in Python using the PyTorch framework, and a pre-trained convolutional neural network model was used.

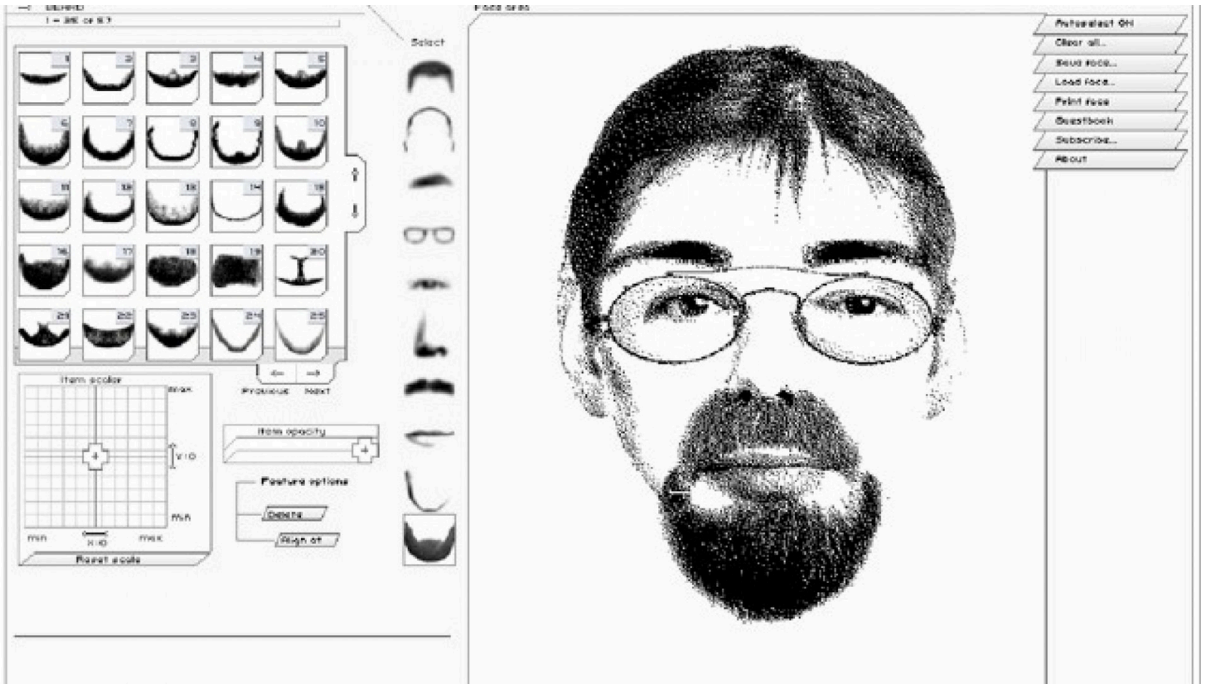
During the work, two surveys were conducted. One was to assess the problem, and the other was to evaluate the results. The first one suggested evaluating 6 photofits on a scale of 1-5/5. The second involved simulating recognition by portrait. From 4 "suspects," the most similar one was chosen.

As a result, the solution satisfied some of the set goals. The results are positive; it was possible to increase the recognition of photorobots and achieve a partially complete but well-drawn image. In the future, it may be possible to continue the work in the form of writing a program using deep learning.

CONTENT

INTRODUCTION

The most popular method in forensics is probably identification by a composite sketch. An artist who quickly draws a portrait of the criminal based on the witness's anxious descriptions is an indispensable element of any detective movie. Although the process itself looks different now, composite sketches still remain a relevantly used method for identifying criminals.



(0.1) - example of software "photo robot"

A photofit is a portrait made up of individual fragments of a person's facial features. In Russia, they are currently mainly used in small towns, villages, and regions. To create such portraits, special software is used, which provides a catalog of a person's facial features, from which a photofit is mechanically assembled. An example of such software is the foreign complex "Faces," which includes 2800 fragments to choose from.

Despite the seemingly extensive set of advantages of creating a photofit using such software, their recognizability is quite low. This was shown by a social survey conducted as part of this research work. Accordingly, an addition to the applied technique is required to improve the results.

So, why was it decided to use computer vision? Let's start with the definition itself. This concept includes technologies and methods that recognize, classify, and generally perform some work with images using machine learning, including neural networks. Computer vision technologies allow for fast, high-quality work on images while providing a unique opportunity to learn from

past experiences. Current software does not have the capability to provide these functions. This solution is also unique: at the moment, nothing similar is used in this field.

Why is it so important to solve the problem of low recognition? It has already been mentioned that composite sketches are used in small federal territories. The reason for this is the lack of a stable video surveillance system. Accordingly, where, say, Moscow has the ability to review the database of video recordings from surveillance cameras, small cities rely entirely on the accuracy of criminal portraits. Therefore, by improving their accuracy, it would be possible to reduce the criminal risk in such areas.

The aim of the following work was to investigate the possibilities of increasing the recognizability of photorobots using computer vision technologies.

1. ANALYSIS

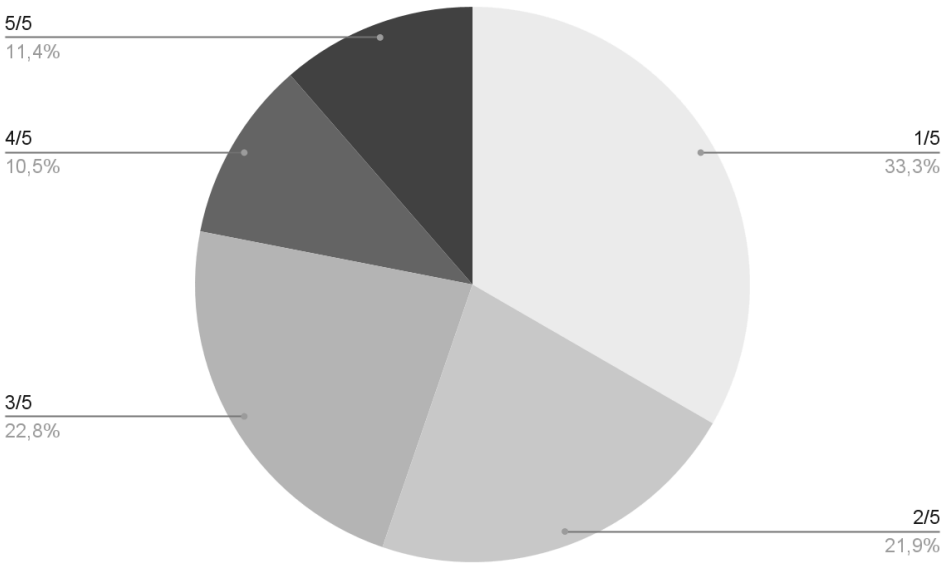
1.1 Assessment of the situation

As an assessment of the situation, a certain list of facts about the current state in this area was compiled:

1. The main method of creating photorobots is software;
2. The target consumer is federal territories of Russia with a small budget;
3. At present, the creation of a photofit is done entirely mechanically;
4. For creating a photofit, the statements of the victim/witness are used;
5. The approximate number of fragments in the complex catalog: about 2000-3000.

1.2 Identifying the problem, social survey

To assess the accuracy of the photofit, a social survey was conducted. Participants were presented with 6 comparisons of photofits of different types with the faces of criminals. They were required to rate their similarity on a scale of 1-5/5 (from one to five out of five). The images were obtained from the internet. The results are shown in the diagram:



(1.1)

As can be seen, only 11.4% rated the similarity as 5/5. At the same time, the majority rated it at the lowest score, 1/5. Such results are extremely unsatisfactory. A significant percentage of low recognition means that

when seeing a criminal in an everyday situation, a person is more likely to lean towards the opinion that he is not wanted.

1.3 Reasons for the problem and possible solutions

So, why were the survey results so low? To understand the reason, an extensive literature review on the topic was conducted. Two main problems with the current solution were identified.

The first of them is that each face is absolutely unique. The entire variety of nose shapes, lip forms, eyebrow thickness, and ear sizes cannot be accommodated in a set of 2000 fragments. This is perhaps the main drawback of the photofit. The too small number of fragments limits the possible combinations of faces. At the same time, increasing their number will not work either: employees will simply get confused among the vast selection. The time to create a photofit will significantly increase.

The second reason is the human factor of the witness. It plays an equally important role, as describing the appearance of the criminal falls to people who have experienced traumatic events. People who have been subjected to an attack, an attempted robbery, or abduction. It turns out that such events directly affect a person's ability to recognize faces. It deteriorates and distorts, causing the witness to become confused in their testimony. Such inaccuracies are difficult to eliminate when creating a composite sketch. And with current technology, it is impossible. It would be good to calculate the average "error" of facial distortion in a person's memory to take it into account in the future.

As a result, four main criteria were established that the solution must meet. They will serve as targets throughout the subsequent work. Criteria:

1. The future solution must be able to provide a complete, realistically rendered image of a person. This requirement is the first and most obvious one. For successful recognition, the quality of the image must be maximized. At the same time, preference will be given to the result that better meets the second condition. It has been found that our brain recognizes a cropped but clear image better than the opposite. The current solution only partially meets this criterion. Composed fragments of faces often create a discontinuous image. Despite the realistic style, the current photofit still resembles a pencil sketch more than a photo. In such a format, it is quite difficult for the brain to perceive the face.

2. The solution must take into account the average error due to the distorted ability of witnesses to remember faces. This condition is the most difficult to fulfill. It can only be achieved by processing extensive past experience. No software for creating composite sketches considers such an important criterion.
3. Minimum execution time. Here, it refers not to the preparation and development of the solution, but to the actual use. An unacceptable execution time is one that takes more than a few minutes. In the context of investigating a case, faster = better.
4. Minimal costs for execution. It is based fundamentally on the target consumer. Previously, we mentioned that this refers to federal territories that do not have the financial means for a quality video surveillance system. Accordingly, the improved solution should be as accessible as possible for any locality.

Gathering the above conditions, one can form an understanding of the necessary solution. The most satisfactory is the application of computer vision technologies. This option is simultaneously inexpensive (the only possible costs are for purchasing the program's execution environment), fast (the result is usually generated in no more than a minute), of high quality, and capable of taking into account the average error values, learning from past experiences.

Two possible types of such a program were identified:

1. Using deep learning technology. Such a model will need to be trained first. That is, a sample from the archive of identified criminals and their sketches must be compiled and "fed" to the program. Supervised learning techniques can be used. The sample should be divided into a test set and a training set, where X represents the sketches and y represents the faces. In other words, the program will learn to bring the sketch as close as possible to how the criminal would look. This solution meets all the established criteria. But, as with everything, there are nuances. First, a prototype of such a solution cannot be created in "amateur" conditions without loss of image accuracy. Obtaining several tens of thousands of examples for samples at home is impossible. Finding 50-100 examples seems realistic, but in that case, the results will be extremely inaccurate. It is also possible to generate several thousand similar examples from a small number of examples, but again, this results in a loss of the program's accuracy.

2. Using the method of image stylization. This solution meets criteria 1, 3, and 4. There is no need to pre-train the model: a ready-made architecture of a convolutional neural network can be used. A photo robot and a style sample are input. The first then takes on the stylistic features of the second, while preserving the facial features. This solution can be implemented without access to any classified data, but it also has a downside. Such a program will not be able to learn from past experiences, so it will not achieve results that strive for perfection. No losses in other criteria are observed: the solution is fast, correct, and free.

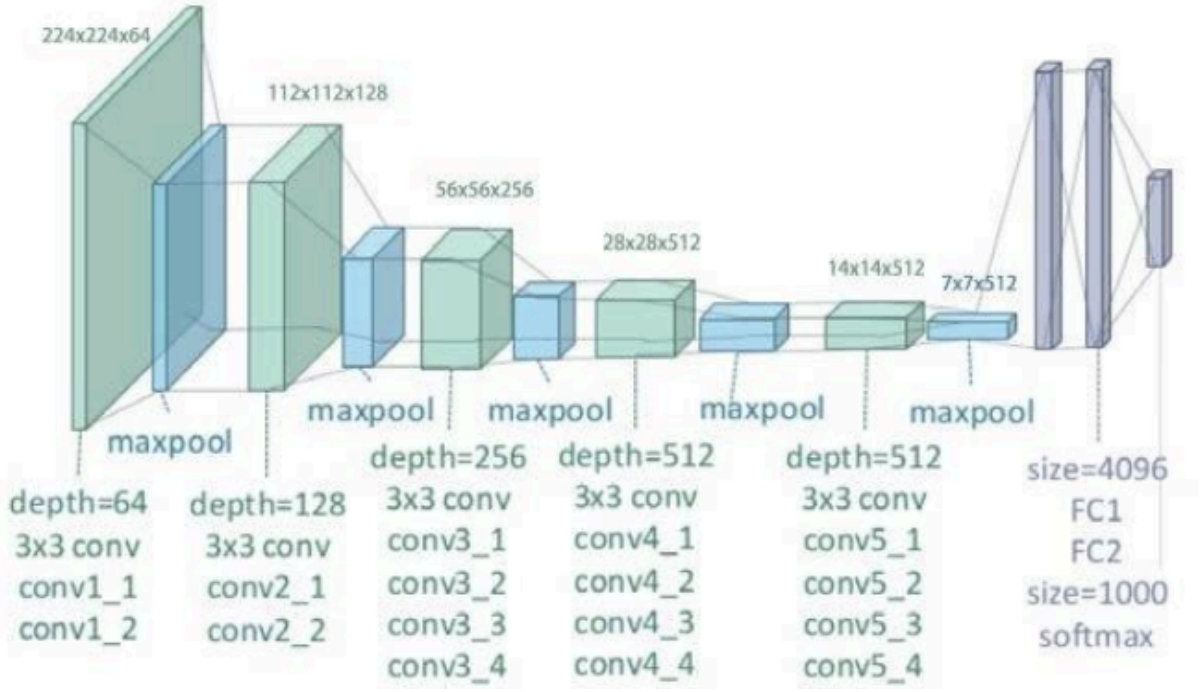
As part of the next work, it was decided to use the second method.

2. WORKING PROCESS

2.1 About the methods used

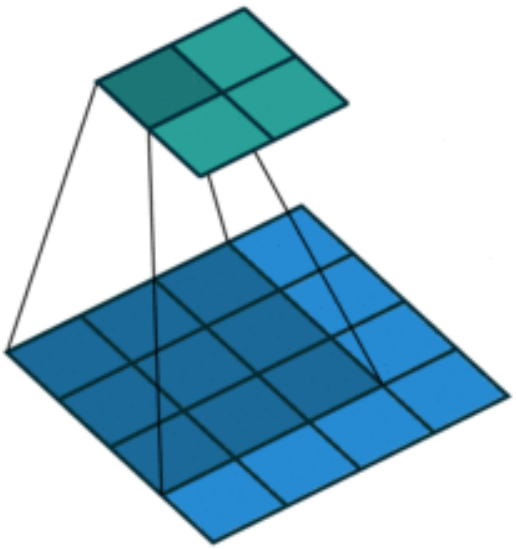
The program is written in Python, using the PyTorch framework. The framework was chosen due to its relevance and extensive functionality. A pre-trained convolutional neural network model vgg-19 was used.

I would like to pay some attention to the architecture of this neural network.



(2.1) - architecture vgg19

The input is an image of size 224 x 224 pixels. It then goes through 2 convolutional layers conv1_1 and conv1_2 with 64 filters. Then a maxpool layer reduces its linear dimensions. And again conv, with increased depth. After repeating such a cycle several times, the result is fed into a fully connected neural network with an input of 4096 neurons using ReLu, and an output of 1000 using softmax.



The animation demonstrating the operation of the convolutional layer is presented on the left.

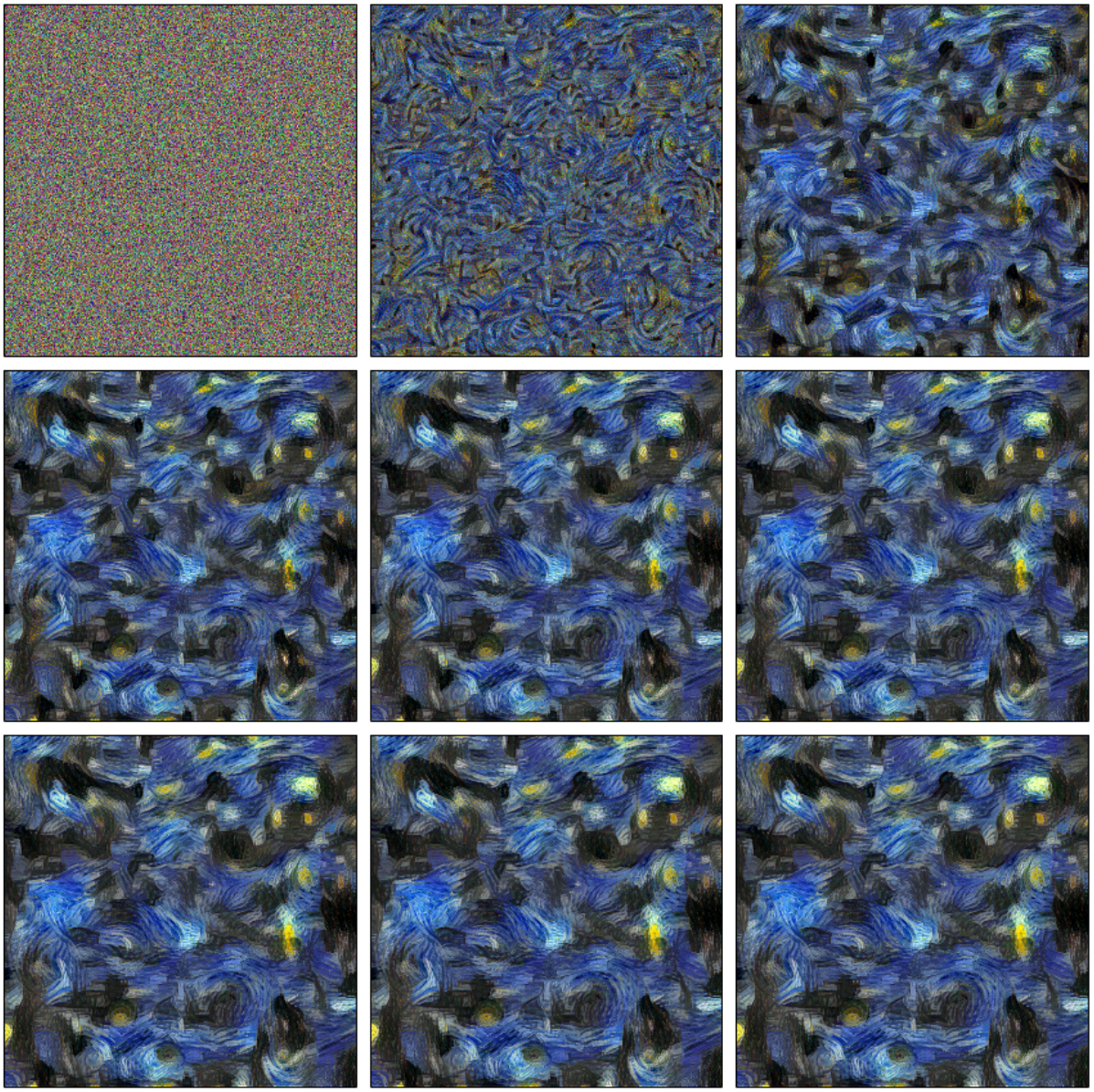
This model has long proven to work excellently for the purposes of

(2.2)

image stylizations, for which it was chosen.

Now a little about the principle of the technology. The original name is Neural Style Transfer. The idea is as follows. We take the original image and consider its pixels as adjustable parameters in the gradient descent algorithm. We choose the quality criterion so that it decreases as the original image approaches the stylized one.

Example of the texture in Van Gogh's paintings:



(2.3)

Results obtained by this method:

(2.4)

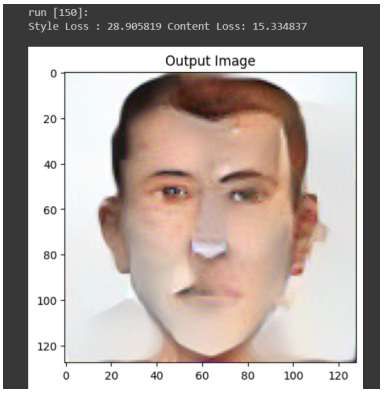
2.2 Parameter selection

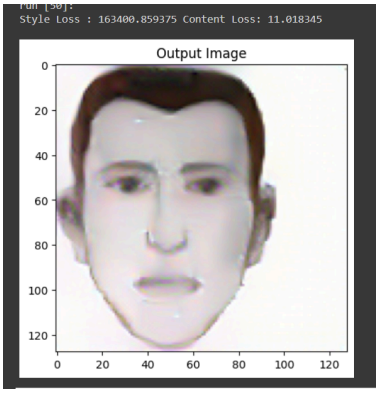
One of the main stages of the work was selecting the parameters. The size of the image, the position of the face relative to the photo, the number of iterations - all of this drastically changed the program's performance. It was necessary to find values that would make the result as realistic as possible.

The most important part was the selection of the style sample. It had to be as universal as possible. That is, possible samples that had rare facial features (unusual eye color, scars, unnatural hair color, albinos, and others) were not considered. Several images were selected for the following tests. In general, about 25 tests were conducted. The best results were shown by two face models:

(2.5.1) *(2.5.2)*

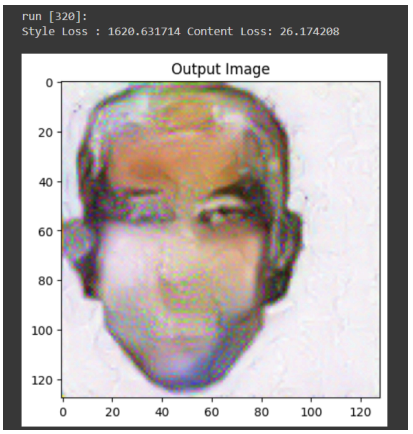
Where (2.5.1) - a face model with averaged features of both sexes. (2.5.2) - a male face model. The images (2) yielded results with a lower error value, while with (2.5.1) the portraits were better detailed but less cohesive. Examples of generation results (2.5.3).





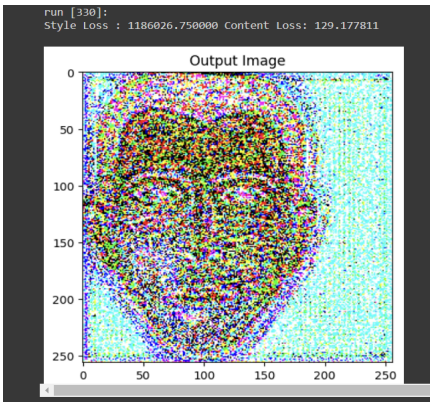
(2.6.1) *(2.6.2)*

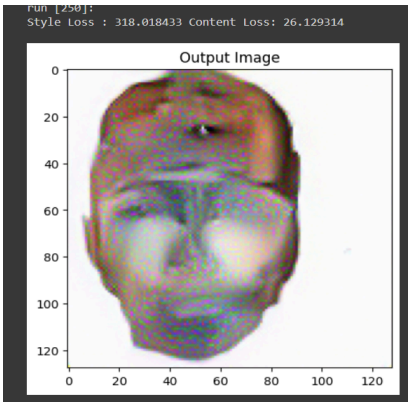
Example (2.5.1):

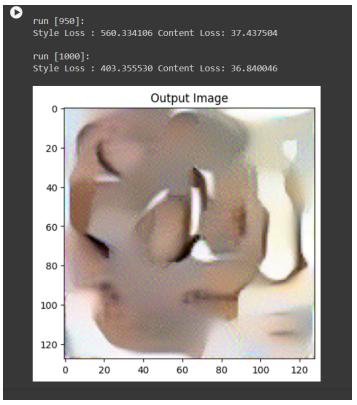


(2.7)

Some unsuccessful attempts:







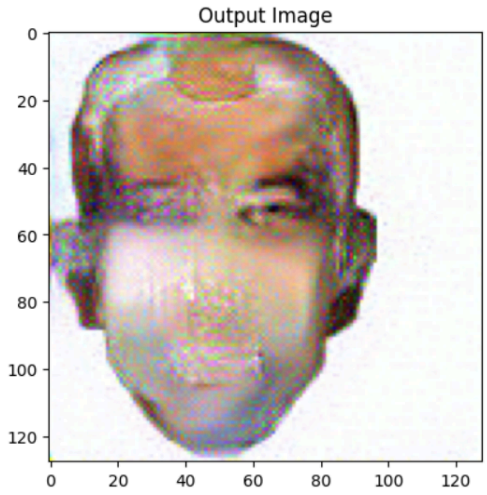
(2.8.1) *(2.8.2)* *(2.8.3)*

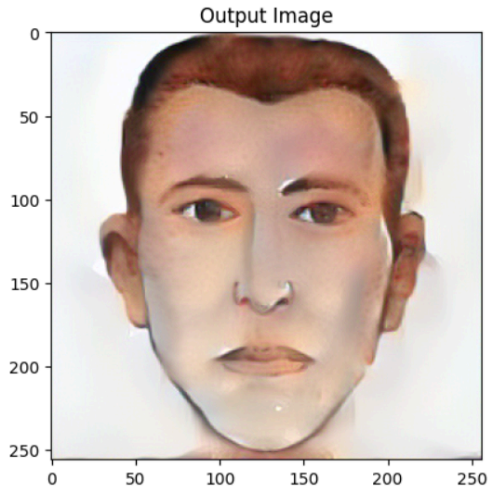
Similar results were obtained with critically high values of the error function. With very low values, the same photorobot was generated, but already colored. Despite the neat appearance of such a result, it was also unsuitable: actual stylization was missing.

The lowest error rates were observed at 150 iterations.

2.3 Results, recognition assessment

After numerous tests, two best results were selected:





(2.9.1) *(2.9.2)*

Parameters (2.9.1):

128x128 pixels

300 iterations

sample style - (1)

Parameters (2.9.2):

256x256 pixels

150 iterations

style sample - (2)

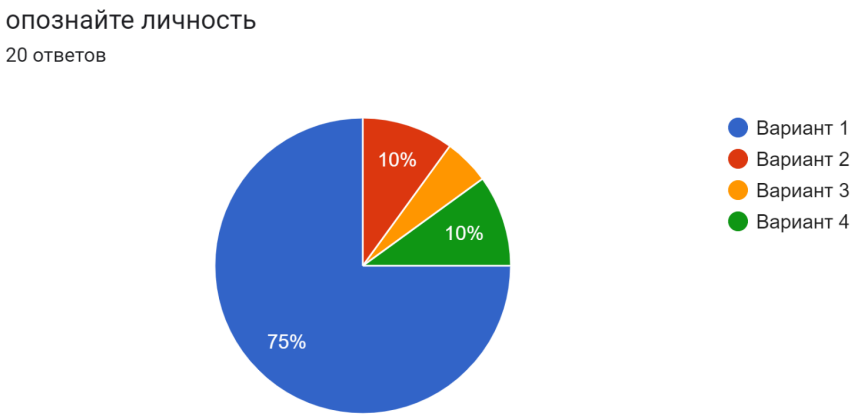
To assess the results, a second social survey was conducted, simulating the process of identifying a criminal. That is, participants were presented with the results of the generation and 4 photographs in the same style.

(2.9.3) *(2.9.4)* *(2.9.5)* *(2.9.6)*

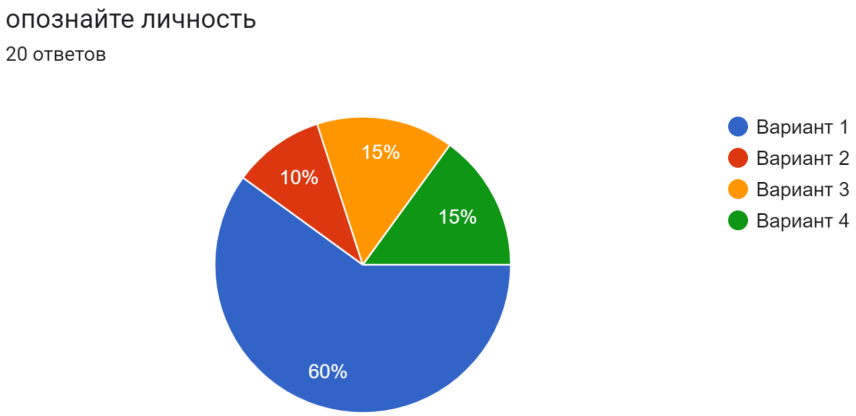
It was necessary to identify the person by the portrait.

Here (2.9.3) is the source. The other photographs were taken from the internet. People were specifically selected from the same time period, with the same black-and-white filter.

Survey results:



(2.9.7)



(2.9.8)

Diagram 2.9.7 belongs to generation 2.9.1, and 2.9.8 belongs to 2.9.2.
The image, which is less cohesive but more realistic, showed better results.

3. CONCLUSIONS

3.1 Recommendations for Practical Application

As part of this scientific work, a widely accessible and fast method has been created to enhance the recognition of a criminal's portrait using image stylization technology. The program can be launched in minutes on any computer and produce results with just the composite image in hand. It is already ready for application in real investigative situations. The solution significantly increases the percentage of image similarity and provides satisfactory results. However, it is less suitable for extensive integration into stable practice, as it does not meet all the criteria set at the beginning of the work.

The second proposed solution requires more time for preparation and training, but allows for the consideration of information from past experience, making it more preferable when officially supplementing it as a function in the "Photobot" software. This arrangement is the most convenient in practice. The employee first creates the photobot as they usually do, and then uses the opportunity for processing with the neural network. As a result, with just an additional minute of waiting, it is possible to maximize the recognizability of the photobot, while meeting all the criteria set at the beginning.

3.2 General conclusion

Ultimately, referring to the highlighted difference between the two solutions, both can be considered satisfactory. They fulfill their specific set of functions in the context of the established goals and are significant.

With their actual implementation, it will be possible to reduce the criminal risk in small towns in Russia and make our country safer. In future work on this project, it makes sense to recreate a second solution option using deep learning technology as much as possible. Conduct similar surveys, note the difference in recognition percentage with the software, and draw appropriate conclusions.

CONCLUSION

After a long period of work, it is finally time to summarize and draw conclusions. By implementing one of the solutions to the problem, it was possible to increase the recognition of the photofit to 75%. Two out of the four established criteria were fully met, one partially. The solution provides a quick, accessible way to edit portraits of criminals. The images come out realistically stylized, but, unfortunately, not complete. Nevertheless, such a result can be considered satisfactory.

LIST OF REFERENCES

1. **V.N. Markivech, A.I. Dudar, T.M. Khusainov The Use of Subjective Portraits in the Work of Law Enforcement Agencies: Theory and Practice // Science. Thought - 2016**
2. Semenova Maria From choosing a nose to catching a crook. How photo robots of criminals are made in Russia // NGC.ru - 2023
3. Pavel Nesterov Stylization of images using neural networks: no mysticism, just math // Habr - 2016
4. Anne Trafton When gender isn’t written all over one’s face // MIT news - 2018
5. Alexis Jacq Neural transfer using torch // PyTorch tutorials
6. Coraline Nehme The effect of post-traumatic stress disorders on the ability to recognize facial expressions // Applied psychology opus

APPLICATION

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

from PIL import Image
import matplotlib.pyplot as plt

import torchvision.transforms as transforms
from torchvision.models import vgg19, VGG19_Weights

import copy
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.set_default_device(device)
# desired size of the output image
imsize = 128

loader = transforms.Compose([
transforms.Resize(imsize),  # scale imported image
transforms.ToTensor()])  # transform it into a torch tensor


def image_loader(image_name):
image = Image.open(image_name)
# fake batch dimension required to fit network's input dimensions
image = loader(image).unsqueeze(0)
    return image.to(device, torch.float)


style_img = image_loader("/content/style.jpg")
content_img = image_loader("/content/input.jpg")

assert style_img.size() == content_img.size(), \
"We need to import style and content images of the same size."
unloader = transforms.ToPILImage()  # reconvert into PIL image

plt.ion()

def imshow(tensor, title=None):
```

```
image = tensor.cpu().clone()  # we clone the tensor to not do changes
on it
image = image.squeeze(0)      # remove the fake batch dimension
image = unloader(image)
plt.imshow(image)
    if title is not None:
        plt.title(title)
    plt.pause(0.001) # pause a bit so that plots are updated


plt.figure()
imshow(style_img, title='Style Image')

plt.figure()
imshow(content_img, title='Content Image')
class ContentLoss(nn.Module):

    def __init__(self, target,):
        super(ContentLoss, self).__init__()
        # we 'detach' the target content from the tree used
        # to dynamically compute the gradient: this is a stated value,
        # not a variable. Otherwise the forward method of the criterion
        # will throw an error.
        self.target = target.detach()

    def forward(self, input):
        self.loss = F.mse_loss(input, self.target)
        return input
def gram_matrix(input):
a, b, c, d = input.size()  # a=batch size(=1)
    # b=number of feature maps
    # (c,d)=dimensions of a function map (N=c*d)

features = input.view(a * b, c * d)  # resize F_XL into \hat F_XL

G = torch.mm(features, features.t())  # compute the gram product

# we 'normalize' the values of the gram matrix
    # by dividing by the number of elements in each feature map.
    return G.div(a * b * c * d)
class StyleLoss(nn.Module):

    def __init__(self, target_feature):
```

```
        super(StyleLoss, self).__init__()
        self.target = gram_matrix(target_feature).detach()

    def forward(self, input):
        G = gram_matrix(input)
        self.loss = F.mse_loss(G, self.target)
        return input
cnn = vgg19(weights=VGG19_Weights.DEFAULT).features.eval()
cnn_normalization_mean = torch.tensor([0.485, 0.456, 0.406])
cnn_normalization_std = torch.tensor([0.229, 0.224, 0.225])

# create a module to normalize input image so we can easily put it in a
# ``nn.Sequential``
class Normalization(nn.Module):
    def __init__(self, mean, std):
        super(Normalization, self).__init__()
        # .view the mean and std to make them [C x 1 x 1] so that they
can
        # directly work with image Tensor of shape [B x C x H x W].
        # B is batch size. C is number of channels. H is height and W
is width.
        self.mean = torch.tensor(mean).view(-1, 1, 1)
        self.std = torch.tensor(std).view(-1, 1, 1)

    def forward(self, img):
        # normalize ``img``
        return (img - self.mean) / self.std
# desired depth layers to compute style/content losses :
content_layers_default = ['conv_4']
style_layers_default = ['conv_1', 'conv_2', 'conv_3', 'conv_4',
'conv_5']

def get_style_model_and losses(cnn, normalization_mean,
normalization_std,
style image, content image,
                               content_layers=content_layers_default,
                               style_layers=style_layers_default):
# normalization module
    normalization = Normalization(normalization_mean,
normalization_std)

# just in order to have an iterable access to or list of content/style
# losses
```

```
    content_losses = []
    style_losses = []

# assuming that ``cnn`` is a ``nn.Sequential``, so we make a new
``nn.Sequential``
# to put in modules that are supposed to be activated sequentially
model = nn.Sequential(normalization)

    i = 0  # increment every time we see a conversation
    for layer in cnn.children():
        if isinstance(layer, nn.Conv2d):
            i += 1
            name = 'conv_{}'.format(i)
        elif isinstance(layer, nn.ReLU):
            name = 'relu_{}'.format(i)
# The in-place version doesn't play very nicely with the
``ContentLoss``
            # and ``StyleLoss`` we insert below. So we replace with
out-of-place
# ones here.
            layer = nn.ReLU(inplace=False)
        elif isinstance(layer, nn.MaxPool2d):
            name = 'pool_{}'.format(i)
        elif isinstance(layer, nn.BatchNorm2d):
            name = 'bn_{}'.format(i)
else:
            raise RuntimeError('Unrecognized layer:
{}'.format(layer.__class__.__name__))

        model.add_module(name, layer)

        if name in content_layers:
            # add content loss:
            target = model(content_img).detach()
            content_loss = ContentLoss(target)
            model.add_module("content_loss_{}".format(i), content_loss)
            content_losses.append(content_loss)

        if name in style_layers:
            # add style loss:
            target_feature = model(style_img).detach()
            style_loss = StyleLoss(target_feature)
            model.add_module("style_loss_{}".format(i), style_loss)
```

```
            style_losses.append(style_loss)

# now we trim off the layers after the last content and style losses
    for i in range(len(model) - 1, -1, -1):
        if isinstance(model[i], ContentLoss) or isinstance(model[i],
StyleLoss):
break

    model = model[:(i + 1)]

    return model, style_losses, content_losses
input_img = content_img.clone()
# if you want to use white noise by using the following code:
#
# ::
#
#    input_img = torch.randn(content_img.data.size())

# add the original input image to the figure:
plt.figure()
imshow(input_img, title='Input Image')
def get_input_optimizer(input_img):
# this line to show that input is a parameter that requires a gradient
    optimizer = optim.LBFGS([input_img])
    return optimizer
def run_style_transfer(cnn, normalization_mean, normalization_std,
                       content_img, style_img, input_img,
num_steps=300,
style_weight=1000000, content_weight=1):
"""Run the style transfer."""
Building the style transfer model..
    model, style_losses, content_losses = get_style_model_and
losses(cnn,
normalization_mean, normalization_std, style_img, content_img

# We want to optimize the input and not the model parameters so we
# update all the requires_grad fields accordingly
input_img.requires_grad_(True)
# We also put the model in evaluation mode, so that specific layers
# such as dropout or batch normalization layers behave correctly.
    model.eval()
    model.requires_grad_(False)
```

```
    optimizer = get_input_optimizer(input_img)

print('Optimizing..')
    run = [0]
    while run[0] <= num_steps:

        def closure():
            # correct the values of updated input image
            with torch.no_grad():
                input_img.clamp_(0, 1)

            optimizer.zero_grad()
            model(input_img)
            style_score = 0
            content_score = 0

            for sl in style_losses:
                style_score += sl.loss
            for cl in content_losses:
                content_score += cl.loss

            style_score *= style_weight
            content_score *= content_weight

            loss = style_score + content_score
            loss.backward()

            run[0] += 1
            if run[0] %  50 == 0:
                print("run {}:".format(run))
                print('Style Loss : {:4f} Content Loss: {:4f}'.format(
style_score.item(), content_score.item()
print()

            return style_score + content_score

        optimizer.step(closure)

    # a last correction...
    with torch.no_grad():
        input_img.clamp_(0, 1)

    return input_img
```

```
output = run_style_transfer(cnn, cnn_normalization_mean,
cnn_normalization_std,
content_img, style_img, input_img

plt.figure()
imshow(output, title='Output Image')

# sphinx_gallery_thumbnail_number = 4
plt.ioff()
plt.show()
```